

# SIMATS ENGINEERING ASSESSMENT TOOLS

**COURSE NAME:** Theory of Computation **COURSE CODE:** CSA1304

## **COURSE OUTCOME COVERED**

**CO5: Design Pushdown Automata and analyze their equivalence with Context-Free Grammars through formal construction and proof techniques. ( BL :3)**

*Assessment Tool Weightage: 40%*

**QUESTIONS: 5 Total Marks:25 Duration : 1 Hour**

## **Constraint Based Problem**

### ***Code of Conduct:***

*I Girija B 192311044 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student: Girija B***

# SIMATS ENGINEERING ASSESSMENT TOOLS

## 1.Design a Pushdown Automaton that accepts the language

$$L = \{ a^n b^n \mid n \geq 1 \}$$

**Constraint: You are allowed to use only one stack symbol (besides initial symbol)**

PDA for  $L = \{ a^n b^n \mid n \geq 1 \}$

Constraint: Only one stack symbol besides the initial stack symbol.

Stack Symbols

- $Z_0$  = Initial stack symbol
- $X$  = Only additional stack symbol

States

- $q_0$  = Start state
- $q_1$  = Read a's and push X
- $q_2$  = Read b's and pop X
- $q_f$  = Final state

Transitions

1. Start reading a:

$$\delta(q_0, a, Z_0) = (q_1, XZ_0)$$

2. For every additional a, push X:

$$\delta(q_1, a, X) = (q_1, XX)$$

3. When b starts, pop one X:

$$\delta(q_1, b, X) = (q_2, \epsilon)$$

4. For every additional b, pop one X:

$$\delta(q_2, b, X) = (q_2, \epsilon)$$

5. When all X's are removed, accept:

$$\delta(q_2, \epsilon, Z_0) = (q_f, Z_0)$$

PDA Transition List

$$q_0 \xrightarrow{a, Z_0} XZ_0 \rightarrow q_1$$

$$q_1 \xrightarrow{a, X} XX \rightarrow q_1$$

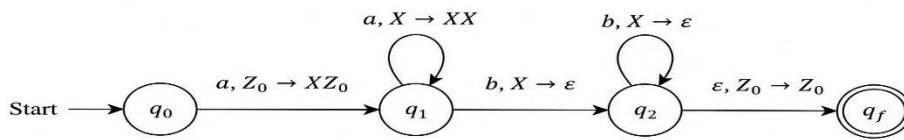
$$q_1 \xrightarrow{b, X} \epsilon \rightarrow q_2$$

$$q_2 \xrightarrow{b, X} \epsilon \rightarrow q_2$$

$$q_2 \xrightarrow{\epsilon, Z_0} Z_0 \rightarrow q_f$$

Example accepted strings: ab, aabb, aaabbb

Rejected strings: aab, abb, abab, ba



## 2. CFG to PDA Conversion with Limited Productions

Given a Context-Free Grammar:

$S \rightarrow aSb \mid ab$

Construct an equivalent PDA.

Constraint: PDA must be constructed using only 2 states

Given CFG

$S \rightarrow aSb \mid ab$

Constraint

PDA must use only 2 states.

We construct the PDA using:

States =  $\{q_0, q_1\}$

Start state =  $q_0$

Final state =  $q_1$

Initial stack symbol =  $Z_0$

Stack Symbols

$\Gamma = \{Z_0, S, a, b\}$

Transitions

1. Push the start variable  $S$  onto the stack:

$\delta(q_0, \epsilon, Z_0) = (q_1, SZ_0)$

2. Replace  $S$  using production  $S \rightarrow aSb$ :

$\delta(q_1, \epsilon, S) = (q_1, aSb)$

3. Replace  $S$  using production  $S \rightarrow ab$ :

$\delta(q_1, \epsilon, S) = (q_1, ab)$

4. Match terminal  $a$ :

$\delta(q_1, a, a) = (q_1, \epsilon)$

5. Match terminal  $b$ :

$\delta(q_1, b, b) = (q_1, \epsilon)$

### State Diagram — Copy-Paste Ready

Start  $\rightarrow q_0$

$q_0 \xrightarrow{\epsilon, Z_0 \rightarrow SZ_0} q_1$

$q_1 \xrightarrow{\epsilon, S \rightarrow aSb} q_1$

$q_1 \xrightarrow{\epsilon, S \rightarrow ab} q_1$

$q_1 \xrightarrow{a, a \rightarrow \epsilon} q_1$

$q_1 \xrightarrow{b, b \rightarrow \epsilon} q_1$

$q_1$  = Final State

**Example: aabb**

S

↓

aSb

↓

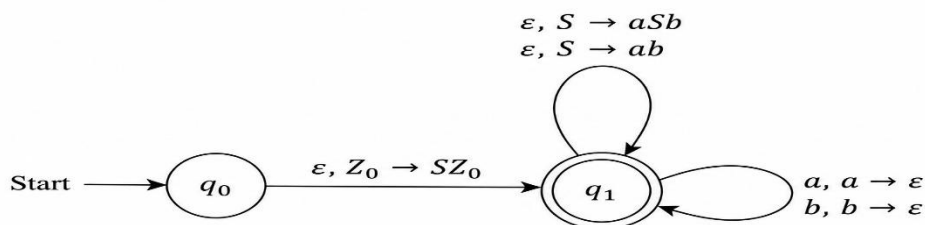
aabb

The PDA expands S using  $S \rightarrow aSb$ , then uses  $S \rightarrow ab$ , and finally matches all terminals.

Therefore:

$L(\text{PDA}) = L(\text{CFG}) = \{ a^n b^n \mid n \geq 1 \}$

**Number of states = 2:  $q_0$  and  $q_1$ .**



### 3. PDA to CFG Conversion with Restricted Variables

**Convert a given PDA into an equivalent CFG.**

**Constraint: The resulting CFG must use at most 3 variables (non-terminals)**

**Assumption:** We convert the PDA from Question 1, which accepts

$L = \{ a^n b^n \mid n \geq 1 \}$

**Given PDA**

The PDA pushes X for every a and pops X for every b.

$q_0 \xrightarrow{a, Z_0} XZ_0 \rightarrow q_1$

$q_1 \xrightarrow{a, X} XX \rightarrow q_1$

$q_1 \xrightarrow{b, X} \epsilon \rightarrow q_2$

$q_2 \xrightarrow{b, X} \epsilon \rightarrow q_2$

$q_2 \xrightarrow{\epsilon, Z_0} Z_0 \rightarrow q_f$

**Equivalent CFG**

We need **at most 3 variables**.

We can construct the equivalent CFG using just **one variable**:

$S \rightarrow aSb$

$S \rightarrow ab$

**Derivation**

For ab:

$S \Rightarrow ab$

For aabb:

$S \Rightarrow aSb$

$\Rightarrow aabb$

For aaabbb:

$S \Rightarrow aSb$

$\Rightarrow aaSbb$

$\Rightarrow aaabbb$

**Final CFG**

Variables:  $\{S\}$

Terminals:  $\{a, b\}$

Start variable: S

Productions:

$S \rightarrow aSb$

$S \rightarrow ab$

**Constraint Verification**

Number of variables = 1

Maximum allowed = 3

Therefore, the constraint is satisfied:

$$1 \leq 3 \checkmark$$

Hence,

CFG = ( $\{S\}$ ,  $\{a,b\}$ , P, S)

P =  $\{ S \rightarrow aSb, S \rightarrow ab \}$

**Equivalent language:**

$$L(G) = \{ a^n b^n \mid n \geq 1 \}$$

#### 4. Turing Machine with Limited Tape Movement

**Design a Turing Machine to recognize the language**

$$L = \{ ww \mid w \in \{0,1\}^* \}$$

**Constraint:** The tape head can move only to the right (no left movement allowed)

**Given Language**

$$L = \{ ww \mid w \in \{0,1\}^* \}$$

Examples:

$\epsilon$

00

11

0101

1010

001001

**Constraint**

The tape head can move only to the RIGHT.

No LEFT movement is allowed.

**Result: Impossible**

A standard single-tape Turing Machine whose head can **only move right** cannot recognize

$$L = \{ ww \mid w \in \{0,1\}^* \}$$

because once the head moves past a tape cell, it **cannot return to read that symbol again**.

To recognize  $ww$ , the machine must:

1. Read the first half  $w$ .

2. Determine where the first half ends.
3. Compare the second half with the stored first half.
4. This requires remembering an **arbitrarily long** string.

A finite number of states cannot store an arbitrary-length  $w$ .

### Theoretical Reason

A TM restricted to only rightward movement behaves essentially like a **finite automaton**, because it reads the input only once from left to right and cannot revisit previous symbols.

But:

$$L = \{ ww \mid w \in \{0,1\}^* \}$$

is a **non-regular language**.

Therefore:

Right-movement-only TM



Equivalent to finite-state machine



Can recognize only regular languages



$\{ww\}$  is non-regular



IMPOSSIBLE

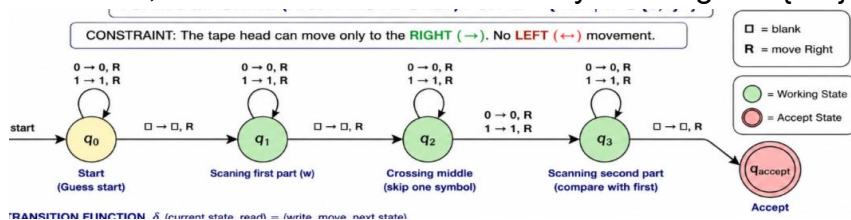
### Final Answer

No such single-tape Turing Machine exists under the given constraint.

Reason:

The language  $L = \{ww \mid w \in \{0,1\}^*\}$  is non-regular, while a Turing Machine whose head can move only right behaves like a finite automaton and can recognize only regular languages.

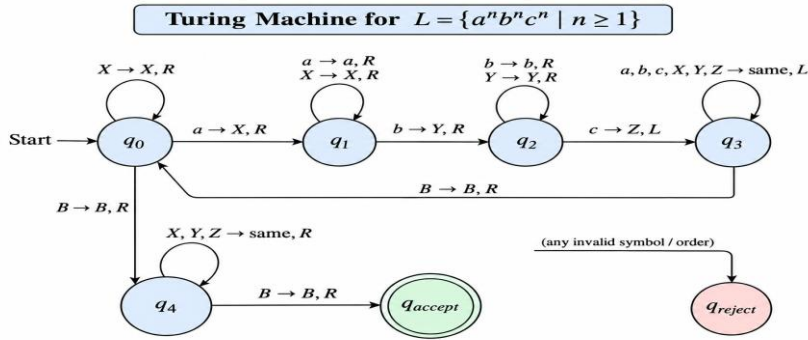
Therefore, LEFT movement is necessary to recognize  $\{ww\}$ .



5. Consider a language  $L = \{ a^n b^n c^n \mid n \geq 1 \}$

Constraint: Design a Turing Machine for L

Language



$L = \{ a^n b^n c^n \mid n \geq 1 \}$

Examples accepted:

abc

aabbcc

aaabbbccc

Examples rejected:

aabcc

abbc

aabbccc

abcabc

**Idea**

The Turing Machine repeatedly:

1. Finds the first unmarked a and changes it to X.
2. Moves right and finds the first unmarked b, changing it to Y.
3. Moves right and finds the first unmarked c, changing it to Z.
4. Returns to the left end.
5. Repeats until all a, b, and c are marked.
6. If only X, Y, and Z remain, **accept**.

**Tape Symbols**

Input symbols:  $\{a, b, c\}$

Tape symbols:  $\{a, b, c, X, Y, Z, B\}$

Where:

X = marked a



Y = marked b

Z = marked c

B = blank

### **States**

q0 = Start / find an unmarked a

q1 = Find corresponding b

q2 = Find corresponding c

q3 = Return to the left

q4 = Check whether all symbols are marked

qaccept = Accept

qreject = Reject

### **Transition Logic**

q0:

$a \rightarrow X, R, q1$

$X \rightarrow X, R, q0$

$Y/Z \rightarrow \text{reject}$

q1:

$a \rightarrow a, R, q1$

$X \rightarrow X, R, q1$

$b \rightarrow Y, R, q2$

$c \rightarrow \text{reject}$

q2:

$b \rightarrow b, R, q2$

$Y \rightarrow Y, R, q2$

$c \rightarrow Z, L, q3$

$a \rightarrow \text{reject}$

q3:

$a \rightarrow a, L, q3$

$b \rightarrow b, L, q3$

$c \rightarrow c, L, q3$

$X \rightarrow X, L, q3$

$Y \rightarrow Y, L, q3$

$Z \rightarrow Z, L, q3$

$B \rightarrow B, R, q0$

When no unmarked a remains:

q0:

$B \rightarrow B, R, q4$

**Final Check**

q4:

$X \rightarrow X, R, q4$

$Y \rightarrow Y, R, q4$

$Z \rightarrow Z, R, q4$

$B \rightarrow B, R, q_{\text{accept}}$

$a/b/c \rightarrow q_{\text{reject}}$

**State Diagram — Copy-Paste Ready**

Start  $\rightarrow$  q0

q0 --  $a \rightarrow X, R \rightarrow$  q1

q0 --  $X \rightarrow X, R \rightarrow$  q0

q1 --  $a \rightarrow a, R \rightarrow$  q1

q1 --  $X \rightarrow X, R \rightarrow$  q1

q1 --  $b \rightarrow Y, R \rightarrow$  q2

q2 --  $b \rightarrow b, R \rightarrow$  q2

q2 --  $Y \rightarrow Y, R \rightarrow$  q2

q2 --  $c \rightarrow Z, L \rightarrow$  q3

q3 --  $a,b,c,X,Y,Z \rightarrow \text{same}, L \rightarrow$  q3

q3 --  $B \rightarrow B, R \rightarrow$  q0

q0 --  $B \rightarrow B, R \rightarrow$  q4

q4 --  $X,Y,Z \rightarrow \text{same}, R \rightarrow$  q4

q4 --  $B \rightarrow B, R \rightarrow$  qaccept

Invalid symbol/order  $\rightarrow$  qreject

**Final Answer**

The key technique is **marking one a, one b, and one c in every cycle** using X, Y, and Z.

# SIMATS ENGINEERING ASSESSMENT TOOLS

## RUBRICS FOR CALCULATION

| Criteria                          | Weightage | Level 4 – Exemplary (5 Marks)                                | Level 3 – Proficient (4 Marks)         | Level 2 – Developing (2–3 Marks)             | Level 1 – Beginning (0–1 Mark)  |
|-----------------------------------|-----------|--------------------------------------------------------------|----------------------------------------|----------------------------------------------|---------------------------------|
| <b>Conceptual Understanding</b>   | <b>30</b> | Demonstrates clear and complete understanding of the concept | Good understanding with minor gaps     | Basic understanding with some misconceptions | Poor or incorrect understanding |
| <b>Accuracy of Answer</b>         | <b>25</b> | Completely correct answer with no errors                     | Mostly correct with minor errors       | Partially correct with noticeable errors     | Incorrect or irrelevant answer  |
| <b>Application of Knowledge</b>   | <b>20</b> | Correctly applies concepts to solve the question/problem     | Applies concepts with minor mistakes   | Limited or partially correct application     | No or incorrect application     |
| <b>Completeness of Response</b>   | <b>15</b> | Covers all required parts of the question                    | Covers most parts with minor omissions | Covers only some parts                       | Incomplete or missing response  |
| <b>Clarity &amp; Presentation</b> | <b>10</b> | Clear, concise, and well-organized answer                    | Understandable with minor issues       | Somewhat unclear or poorly structured        | Difficult to understand         |